

FIAP GRADUAÇÃO

| Oracle

- ▶ Introdução
- ▶ Implementação
- ▶ Modulando com funções
- ▶ Exercícios

Introdução

- ▶ O Python é uma linguagem de programação de propósito geral, de alto nível, interpretada, de tipagem dinâmica e com sintaxe fácil de ler.
- ▶ O Oracle é um sistema de gerenciamento de banco de dados relacional (RDBMS) de propriedade da Oracle Corporation.
- ▶ O Python pode ser usado para acessar bancos de dados Oracle de várias maneiras, incluindo:
 - ▶ O pacote `oracledb`
 - ▶ O pacote `cx_Oracle`

O pacote oracledb

- ▶ O pacote oracledb é um pacote Python que fornece uma interface para o Oracle Database.
- ▶ é um pacote nativo, o que significa que ele é compilado para a plataforma específica em que está sendo usado.
- ▶ segundo a documentação é um upgrade do pacote cx_Oracle
- ▶ para instalar no computador, execute o comando `pip install oracledb` ou `pip3` se você está no MAC ou Linux

Exemplo de código de consulta

O seguinte código mostra como usar o pacote `oracledb` para conectar-se ao Oracle Database e recuperar os dados da tabela `T_MENSAGEM`:

```
1 import oracledb
2
3 conn = oracledb.connect(user='pf0313', password='professor
    #23', dsn="oracle.fiap.com.br/orcl")
4 print("Database version:", conn.version)
5
6 cur = conn.cursor() # Criar um cursor
7 cur.execute('SELECT * FROM T_MENSAGEM') # Executar uma
    consulta
8
9 rows = cur.fetchall() # Recuperar os resultados da consulta
10 for reg in rows:
11     print(reg)
12
13 cur.close() # Fechar o cursor
14 conn.close() # Fechar a conexao
```

Métodos fetch

- ▶ `fetchall()` - recupera todos os registros da consulta, tome cuidado se a quantidade de registros for muito grande pode faltar memória além de demorar muito tempo
- ▶ `fetchone()` - recupera um registro da consulta
- ▶ `fetchmany(size)` - recupera uma quantidade `size` de registros ou se nenhum valor for passado como parâmetro recupera um número default de registros
- ▶ veja abaixo um exemplo que recupera todos os registros da consulta usando o método `fetchmany()`

```
1 while True:
2     regs = cursor.fetchmany()
3     if not regs:
4         break
5     for reg in regs:
6         print(reg)
```

Inserir, atualizar e excluir

O seguinte código mostra como usar o pacote `oracledb` para inserir, atualizar e excluir registros em uma tabela:

```
1 conn = oracledb.connect(user='pf0313', password='professor
   #23', dsn="oracle.fiap.com.br/orcl")
2 cur = conn.cursor()
3 # Inserir um novo registro
4 cur.execute("INSERT INTO EMPREGADO (numero, nome, cargo,
   data_contratacao, salario, departamento) VALUES (:num, :
   nome, :cargo, to_date(:data, 'YYYY-MM-DD'), :sal, :depto
   )", {'num': 7839, 'nome': 'SCOTT', 'cargo': 'ANALYST', '
   data': '1987-04-19', 'sal': 3000, 'depto': 'TI'})
5
6 #continua no proximo eslaide
```

Inserir, atualizar e excluir - Continuação

```
1  # Atualizar um registro existente
2  cur.execute('UPDATE EMPREGADO SET salario = :sal WHERE
              numero = :num', {'sal': 3500, 'num': 7839})
3
4  # Excluir um registro
5  cur.execute('DELETE FROM EMPREGADO WHERE numero = :num', {'
              num': 7839})
6
7  #commit dos dados no banco sempre que executar comandos que
      modificam os dados
8  conn.commit()
9  cur.close()
10 conn.close()
```


Considerações

- ▶ não esquecer de fechar o cursor e depois fechar a conexão
- ▶ comandos de alteração, inserção e delete devem ser finalizados com o método `commit` da conexão
- ▶ caso contrário, nenhuma informação será gravada
- ▶ temos também o método `rollback()` da conexão que desfaz todas as alterações no registros do banco desde a abertura da conexão
- ▶ em geral, executamos um `rollback()` quando algum erro acontece e queremos desfazer das modificações

Outra forma de abrir conexões e cursores

- ▶ podemos abrir conexões e cursores usando o comando `with`
- ▶ abrindo com `with` não precisamos fechar explicitamente eles
- ▶ no fim do bloco `with` os recursos são fechados de forma automática
- ▶ veja um exemplo:

```
1 with oracledb.connect(user='pf0313', password='professor#23',  
2    , dsn='oracle.fiap.com.br/orcl') as conn:  
3     with conn.cursor() as cur:  
4         cur.execute("insert into categoria values (:id, :  
                    nome)", {'id':1, 'nome':'alimento'})  
5     conn.commit()
```

Conexões em forma de função

- ▶ como as conexões são muito utilizadas, é interessante colocar na forma de função
- ▶ veja o código abaixo:

```
1 def get_conexao() -> conn:
2     return oracledb.connect(user='user', password='pwd',
3                               dsn='dbhost.example.com/orcl')
4
5 #exemplo de uso
6 if __name__ == "__main__":
7     with get_conexao() as conn:
8         with conn.cursor() as cur:
9             #execucao do seu codigo SQL
10            conn.commit() #caso nao seja uma consulta
```

- ▶ assim, qualquer alteração de usuário ou endereço do banco você altera em um único local

Passagem de parâmetros para os comandos SQL

- ▶ a fim de evitar comandos nocivos através do SQL Injection, passamos parâmetros para os comandos SQL através de variáveis vinculadas
- ▶ existem algumas formas de passarmos os valores para os comandos SQL
- ▶ vamos focar na passagem por nome usando para isso um dicionário

```
1 def insere_mensagem(msg: dict):
2     with get_conexao() as conn:
3         with conn.cursor() as cur:
4             sql = "insert into t_mensagem(subject, content,
5                     to, from) values (:subject, :content, :to, :
6                     from)"
7             cur.execute(sql, msg)
8             conn.commit()
```

- ▶ onde msg = {'subject': 'Prova', 'content': 'Estude banco de dados', 'to': 'alunos@fiap.com.br', 'from': 'profeduardo@fiap.com.br'}

Exercícios

1. Crie um programa em python que faz a inserção de um filme, peça os dados necessários para o usuário e faça o insert no banco de dados
2. Coloque vários filmes em sua tabela (pode ser através do SQL Developer).
3. Faça um programa em Python que faz uma consulta por alguma informação do filme.
4. Transforme o insert do exercício 1 em uma função. Aproveite para fazer o mesmo com a consulta e acrescente funções para alterar e apagar filmes.